

Technical Documentation

Electromechanical Simulation and Multi-Architecture Gravitational Energy Storage Analysis Software

May 11, 2026

Abstract

This document specifies the mathematical models, software architecture, file responsibilities, and data products of the MATLAB-based simulation package. Two complementary layers are described: (1) an ordinary-differential-equation (ODE) electromechanical core for a lumped hoist–drivetrain–motor–electrical load plant, and (2) lumped cycle models for four gravitational storage architectures, integrated in the time domain for comparative energy metrics. All quantities are expressed in SI units unless noted. The objective is complete traceability from physical assumptions to source files and outputs.

Keywords: lumped-parameter model, round-trip efficiency, MATLAB, ODE45, gravitational energy storage, regenerative hoist.

Contents

1	Scope and modeling philosophy	3
2	Repository structure and execution	3
3	System architecture and data flow	3
3.1	Physical signal path (conceptual)	4
4	Layer 1: ODE electromechanical core	4
4.1	system_parameters.m	4
4.2	main_simulation.m	4
4.3	solvers/system_ode.m	5
4.4	models/mass_drum_model.m	5
4.5	models/sprocket_model.m, gearbox_model.m, bevel_gear_model.m	5
4.6	models/motor_model.m	5
4.7	models/load_model.m	5
4.8	utils/reflect_inertia.m	6
4.9	analysis/energy_analysis.m	6
4.10	analysis/validation_checks.m	6
4.11	analysis/plot_results.m	6
5	Layer 2: Architecture cycle models	6
5.1	scripts/plot_dual_weight_results.m	6
5.2	scripts/plot_variable_counterweight_results.m	6
5.3	scripts/plot_buoyancy_gravity_results.m	7
5.4	scripts/plot_halbach_gravity_results.m	7
5.5	scripts/analysis_compare_systems.m	7
5.6	scripts/run_tiered_gravity_analysis.m and get_gravity_system_data	7

6	Supporting and presentation scripts	7
7	Equation summary (cross-reference)	8
8	File index	8
A	Operational phase abstraction (example)	8
B	Extended narrative: <code>main_simulation.m</code> execution order	9
C	Symbolic conventions	9

1 Scope and modeling philosophy

The software serves two purposes. First, it integrates a *single-degree-of-freedom* rotational model coupled to a DC-equivalent motor and electrical load, yielding time histories consistent with Newton–Euler dynamics and Kirchhoff–type electrical constraints. Second, it evaluates *architecture-level* energy flows for four conceptually distinct systems by prescribing smooth velocity profiles over operational phases and applying scalar efficiencies and auxiliary power terms, then integrating electrical power.

Lumped-parameter modeling replaces spatially distributed phenomena (fluid fields, magnetic finite elements, multi-segment rope elasticity) with effective masses, phase durations, efficiencies, and bounded auxiliary powers. This is standard when the engineering question is comparative energy accounting under identical gravitational potential budgets rather than subsystem field resolution.

2 Repository structure and execution

The project root contains `setup_path.m`, which adds the repository and subfolders to the MATLAB path. Primary directories:

- `models/` — component functions called by the ODE.
- `solvers/` — `system_ode.m`.
- `analysis/` — `energy_analysis.m`, `validation_checks.m`, `plot_results.m`.
- `scripts/` — runnable drivers, architecture plot scripts, statistical comparison, tiered analysis.
- `output/` — generated tables, figures, and text summaries (when scripts write there).
- `docs/` — supplementary Markdown references (optional reading alongside this PDF).

The ODE simulation is invoked by calling `main\simulation(params)` with `params = system\_parameters` or a modified structure. Architecture scripts are standalone functions (e.g. `plot\_dual\_weight\_results`).

3 System architecture and data flow

Figure 1 shows the layered data flow between parameter files, numerical integrator, component models, and post-processing. Physical power flows along the mechanical chain; algebraic variables couple the electrical port.

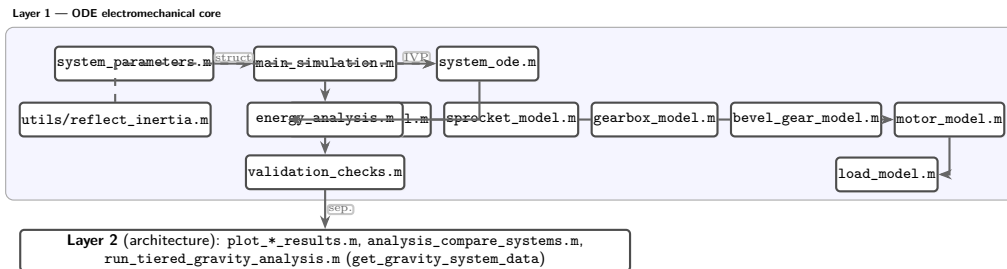


Figure 1: Layered software and physical signal flow. Layer 1 couples gravitational torque, mechanical train losses, motor electrical equations, and load. Layer 2 uses prescribed kinematics and lumped efficiencies for cross-architecture comparison; it does not call the ODE unless explicitly integrated elsewhere.

3.1 Physical signal path (conceptual)

Figure 2 abstracts the plant represented by Layer 1: a point mass moves vertically in one dimension; motion is coupled without slip to a drum; the drum drives a fixed-ratio mechanical train to a rotary electrical machine; the machine terminals connect to an electrical load model.

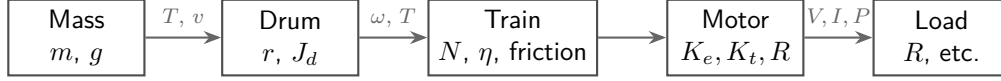


Figure 2: Conceptual electromechanical signal path for Layer 1 (single equivalent vertical DOF coupled to one electrical port).

4 Layer 1: ODE electromechanical core

4.1 `system\_parameters.m`

Returns a MATLAB structure grouping: physical constants (m , g , drum radius and inertia, sprocket ratio and inertia, initial height and velocity); gearbox ratios and inertias; bevel ratio and inertia; motor catalog data (K_v , hence K_e), winding resistance and inductance, rotor inertia, current limit, optional loss coefficients; per-stage efficiency and Coulomb/viscous friction torques; electrical load type and parameters; ODE solver tolerances and flags (`include\_electrical\_dynamics`, `stop\_at\_bottom`); energy tracking flags; optimization sweep bounds.

Motor voltage constant:

$$K_e = \frac{60}{2\pi K_v} \quad (\text{units: V s rad}^{-1}), \quad K_t = K_e \quad (\text{SI ideal equality}). \quad (1)$$

Every numeric field is documented inline in the source file with units.

4.2 `main\_simulation.m`

Initialization. Converts initial linear speed to drum angular speed $\omega_0 = v_0/r_{\text{drum}}$. Computes total kinematic ratio N_{tot} from sprocket, gearbox product, and bevel. Sets initial motor speed $\omega_{m,0} = \omega_0/N_{\text{tot}}$. Builds initial state: $[\theta_0, \omega_0, \omega_{m,0}]$ or, if electrical dynamics enabled, appends initial current from `load\_model`.

Integration. Wraps `system\_ode` for the solver; sets relative and absolute tolerances, maximum step, and an event function that fires when height reaches zero downward to terminate the simulation when configured.

Post-processing loop. For each accepted time sample, calls `system\_ode` again with `nargout` requesting component outputs, then `energy\_analysis` to build per-step power and energy metrics.

Energy integration. Trapezoidal accumulation of electrical power and mechanical and electrical loss powers between consecutive samples.

Extraction. Assembles time series for angles, speeds, currents (from state or reconstructed), height, linear speed, mechanical and electrical powers, cumulative energies, and instantaneous efficiency.

Conservation check. Compares initial gravitational energy mgh_0 to final electrical energy plus integrated losses plus residual kinetic energy (linear mass term plus rotational contributions from drum, sprocket, gearbox, bevel, motor rotor) to report an energy closure error.

Output structure. Packages all series, parameters copy, timing, final scalar metrics, and passes results to `validation\_checks`.

Local helpers. `system\_ode\_wrapper` adapts ODE signature; `mass\_reached\_bottom` implements the terminal event on height.

4.3 solvers/system_ode.m

State definition. θ drum angle; ω drum speed; ω_m motor speed; optional I .

Electrical branch without inductance dynamics. Iterative fixed-point solution for I from back-EMF $V_{\text{emf}} = K_e \omega_m$, terminal voltage, and `load\_model`, with current clamped to I_{max} .

Bottom stop. If $h \leq 0$ and stopping is enabled, returns zero derivatives and zeroed component outputs for a clean termination state.

Mechanical chain. Sequentially applies `mass\_drum\_model`, `sprocket\_model`, `gearbox\_model`, `bevel\_gear\_model`, `motor\_model`. Re-evaluates current from terminal voltage when not using full electrical state.

Inertia aggregation. Sums reflected inertias to an effective J at the drum using J/N^2 relationships consistent with rigid gearing.

Kinematic enforcement. Sets $\omega_m = \omega/N_{\text{tot}}$ and recomputes motor outputs for consistency.

Torque balance. Reflects motor electromagnetic torque to the drum with ratio N_{tot} and forms net torque $T_{\text{grav}} - T_{m,\text{ref}}$; angular acceleration $\alpha = T_{\text{net}}/J_{\text{tot}}$.

State derivatives. $\dot{\theta} = \omega$, $\dot{\omega} = \alpha$, $\dot{\omega}_m = \alpha/N_{\text{tot}}$; optional dI/dt from an L - R simplification when enabled. Optional scaling of $\dot{\omega}_m$ if current exceeds limit (simplified limiter).

Component output struct. Passes heights, velocities, torques, per-stage loss powers, electrical power, motor loss power, currents, voltages, and motor speed for downstream energy accounting.

4.4 models/mass_drum_model.m

Implements:

$$h = \max(0, h_0 - r\theta), \quad (2)$$

$$v = r\omega, \quad (3)$$

$$T_{\text{grav}} = mgr, \quad (4)$$

$$J_{\text{eff}} = J_{\text{drum}} + mr^2. \quad (5)$$

4.5 models/sprocket_model.m, gearbox_model.m, bevel_gear_model.m

Common pattern per stage: output speed $\omega_{\text{out}} = \omega_{\text{in}}/N$; input power $P_{\text{in}} = T_{\text{in}}\omega_{\text{in}}$; efficiency-scaled power $P' = \eta_{\text{tot}}P_{\text{in}}$ with $\eta_{\text{tot}} = \eta^n$ for multi-stage gearbox; friction torque on input shaft combined with viscous term; net power; output torque from $P_{\text{net}}/\omega_{\text{out}}$ when rotating; power loss $P_{\text{in}} - T_{\text{out}}\omega_{\text{out}}$ with non-negativity and sanity clamp; reflected inertia bookkeeping (gearbox sums input inertia plus output and intermediate inertias scaled by cumulative ratio squared).

4.6 models/motor_model.m

$$V_{\text{emf}} = K_e \omega_m, \quad V_t = V_{\text{emf}} - IR, \quad (6)$$

$$T_{\text{em}} = -K_t I, \quad P_{\text{elec}} = V_t I, \quad (7)$$

$$P_{\text{Cu}} = I^2 R, \quad P_{\text{loss}} = P_{\text{Cu}} + P_{\text{core}} + P_{\text{bearing}}(\omega_m). \quad (8)$$

Bearing friction uses Coulomb plus viscous torque multiplied by speed for power; torque is adjusted to include bearing torque directionally.

4.7 models/load_model.m

Branches: resistive $I = V_t/R$; battery charging with internal resistance and voltage threshold; constant electrical power demand mapped to $I = P_{\text{in}}/V_t$ with efficiency; custom function handle. Non-negative current clamp and global I_{max} clamp.

4.8 utils/reflect_inertia.m

$$J_{\text{ref}} = J/N^2. \quad (9)$$

4.9 analysis/energy_analysis.m

Computes $E_g = mgh$; kinetic contributions at drum and motor; sums mechanical loss powers from sprocket, gearbox, bevel; electrical loss power from motor model; $P_{\text{mech,drum}} = T_{\text{grav}}\omega$; builds nested per-stage power accounting structure; computes instantaneous stage efficiencies and overall $\eta_{\text{system}} = P_{\text{elec}}/P_{\text{mech,drum}}$ with guards against division by zero; returns a complete struct for plotting and tables.

4.10 analysis/validation_checks.m

Recomputes end-of-simulation energy balance; checks mass, velocity, current, and voltage ranges for unit sanity; compares peak velocity to free-fall bound $\sqrt{2gh_0}$; validates efficiency bounds; scans for NaN/Inf in state; optionally warns on large numerical acceleration spikes.

4.11 analysis/plot_results.m

Optional visualization of `main\_simulation` outputs (see file header comments); not required for numerical core.

5 Layer 2: Architecture cycle models

Each script below defines phase times, smooth normalized velocity profiles $v(t)$, mechanical power where applicable, electrical generation and consumption powers, cumulative energies via trapezoidal integration (`cumtrapz`), and multiple figures (time series, energy summaries, summary tables). Remaining lines configure default graphics colors, axis labels, legends, font names, grid styling, `uitable` summary panels, and restore MATLAB default graphics properties at exit. Those presentation blocks do not alter the underlying power definitions.

5.1 scripts/plot_dual_weight_results.m

Physical story. Elevator-style cab and counterweight with opposite motion; net imbalance mass m_{net} .

Mechanical power. $P_{\text{mech}} = m_{\text{net}}g(-v_{\text{cab}})$ with descent and ascent windows.

Electrical mapping. Generation interval: $P_{\text{elec}} = \eta_{\text{gen}} \max(P_{\text{mech}}, 0)$. Ascent motoring: $P_{\text{elec}} = -|P_{\text{mech}}|/\eta_{\text{drive}}$ (negative for consumption convention).

Additional series. Instantaneous regeneration efficiency curve for plotting during descent only; synthetic bounded motor current shape for visualization.

5.2 scripts/plot_variable_counterweight_results.m

Discrete mass schedule. Integer $n_{\text{modules}}(t)$ controlling attached modules; total mass $m_{\text{tot}}(t) = m_{\text{cabled}} + n_{\text{modules}}m_{\text{module}}$.

Descent kinematics. Smooth $h(t)$ and $v(t)$ from bottom to top of stroke.

Main generation. $P_{\text{mech,main}} = m_{\text{tot}}g(-v)$ on descent; $P_{\text{main,elec}} = \eta_{\text{main,gen}} \max(P_{\text{mech,main}}, 0)$.

Secondary generation event. Gaussian-shaped power burst timed at module-count transition to represent localized harvesting.

Dock buffer discharge. Positive electrical power waveform during dock phase added to generation total.

Ascent motor. Ideal counterweight mass $m_{\text{ideal}} = m_{\text{cabled}} + m_{\text{module}}$; net load force $\max(0, m_{\text{tot}}g - \lambda m_{\text{ideal}}g)$ with $\lambda < 1$ representing imperfect balance; mechanical ascent power is net load times velocity; electrical draw divides by η_{drive} with a minimum power floor.

Metrics. Cumulative consumed (motor) and generated (main + module + supercap branch); round-trip efficiency ratio of totals.

5.3 scripts/plot_buoyancy_gravity_results.m

Lift phase. Chamber fill state $f(t) \in [0, 1]$; motor electrical power reduced as fill increases using a scalar `buoyancy\_fraction` representing effective buoyant support of weight.

Water phase. Weight held at top height; pump electrical load active in a defined temporal window with sinusoidal variation.

Drop phase. Velocity profile yields $P_{\text{gen}} = \eta_{\text{gen}}mg(-v)$.

Totals. Consumption is motor plus pump powers integrated; generation integrates P_{gen} .

5.4 scripts/plot_halbach_gravity_results.m

Lift. Thrust force scaled with mg and mild variation; mechanical power Fv ; electrical lift power divides by η_{lift} ; constant hoist-hold electrical power during lift and hold to represent rope tension maintenance.

Drop. Same gravitational electrical generation form as other systems.

Visualization proxies. Sinusoidal “phase current” for qualitative multi-phase ripple; not an inverter switching model.

5.5 scripts/analysis_compare_systems.m

Reconstructs the same kinematic and power structures as the architecture plot scripts, then executes n_{rep} replicates with bounded random perturbations of efficiency-like scalars for uncertainty bands. Computes per-replicate energies and efficiencies; performs one-way ANOVA-style sum-of-squares breakdown for efficiency and net energy; writes textual conclusion paths under `output/` when configured; generates comparative figures.

Consistency note. Variable-counterweight statistics in this file may include additional high-power auxiliary consumption terms during dock/load segments relative to the standalone variable-counterweight plot script; numerical results therefore correspond to the analysis script’s explicit definitions.

5.6 scripts/run_tiered_gravity_analysis.m and get_gravity_system_data

The nested function `get\_gravity\_system\_data` reproduces the architecture comparison mathematics with an additional per-system multiplicative scale on consumption energies. Tiered figures and text tables summarize round-trip metrics, loss allocations by fixed fractional split of total loss (for reporting), and cost scaling assumptions.

6 Supporting and presentation scripts

- `scripts/plot\_figure7\_figure8\_presentation.m` — Static poster values for energy and efficiency bar charts; exports PNG under `output/`.
- `scripts/plot\_figure4\_cycles\_vs\_height.m` — Parametric curves for scaling behavior vs height.
- `scripts/generate\_backboard\_data\_tables.m` — CSV generation for display tables.
- `scripts/run\_example.m`, `scripts/diagnose\_issues.m`, `scripts/optimize\_gearbox.m` — Example, diagnostic, and parameter sweep utilities invoking `main\_simulation`.

7 Equation summary (cross-reference)

Table 1 collects recurring forms.

Table 1: Recurring physical and electrical relations.

Quantity	Relation
Gravitational power (translation)	$P \approx mg(-v)$ with sign per convention
Drum height	$h = \max(0, h_0 - r\theta)$
Motor back-EMF	$V_{\text{emf}} = K_e \omega_m$
Terminal voltage	$V_t = V_{\text{emf}} - IR$
Electrical power	$P_{\text{elec}} = V_t I$
Trapezoidal energy	$E_n = E_{n-1} + \frac{1}{2}(P_n + P_{n-1})(t_n - t_{n-1})$
Reflected inertia	$J_{\text{ref}} = J/N^2$

8 File index

File	Role
<code>system_parameters.m</code>	Central SI parameter struct for ODE core.
<code>main_simulation.m</code>	ODE integration driver, energy integration, results struct, terminal event.
<code>solvers/system_ode.m</code>	Coupled mechanical–electrical right-hand side.
<code>models/mass_drum_model.m</code>	$h, v, T_{\text{grav}}, J_{\text{eff}}$.
<code>models/sprocket_model.m</code>	Sprocket ratio, hybrid losses, reflected inertia.
<code>models/gearbox_model.m</code>	Multi-stage ratio, compounded efficiency, friction, inertia reflection.
<code>models/bevel_gear_model.m</code>	Axis redirection stage with losses.
<code>models/motor_model.m</code>	DC-equivalent motor/generator port.
<code>models/load_model.m</code>	Terminal current laws.
<code>utils/reflect_inertia.m</code>	J/N^2 helper.
<code>analysis/energy_analysis.m</code>	Instantaneous energy metrics and stage efficiencies.
<code>analysis/validation_checks.m</code>	Conservation and plausibility tests.
<code>scripts/plot_dual_weight_results.m</code>	Dual-mass cycle model and figures.
<code>scripts/plot_variable_counterweight_results.m</code>	Variable-mass cycle model and figures.
<code>scripts/plot_buoyancy_gravity_results.m</code>	Buoyancy-style multi-phase cycle.
<code>scripts/plot_halbach_gravity_results.m</code>	Linear-motor lift + hoist + drop cycle.
<code>scripts/analysis_compare_systems.m</code>	Replicated statistics and comparative figures.
<code>scripts/run_tiered_gravity_analysis.m</code>	Tiered summaries; contains <code>get_gravity_system_data</code> .
<code>setup_path.m</code>	MATLAB path configuration.

A Operational phase abstraction (example)

Figure 3 illustrates how architecture scripts partition time into phases with distinct power terms. The numerical values of durations and coefficients reside in the MATLAB sources and may be replaced by measured cycle data.

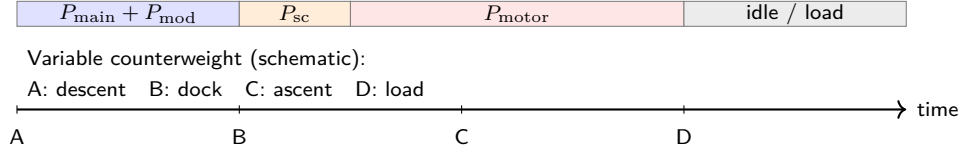


Figure 3: Representative power-channel allocation across cycle phases (not to scale).

B Extended narrative: `main\_simulation.m` execution order

The function entry optionally supplies `params`; otherwise it calls `system\_parameters`. Initial height and drum radius define the geometric constraint. Initial linear velocity converts to drum angular velocity. The motor initial speed follows the fixed kinematic ratio. The initial state vector is assembled; if full electrical dynamics are disabled, current is not a state variable at $t = 0$ beyond what the first `system\_ode` evaluation will infer.

An anonymous function wraps `system\_ode` for the MATLAB ODE solver interface. Solver options include tolerances, maximum step, and an event function that monitors rope-paid height derived from θ . The solver name is selected from `params.simulation.solver`.

After integration, each stored state is reprocessed: `system\_ode` is invoked with the same arguments to recover the component struct consistent with that state, then `energy\_analysis` computes the public energy metrics struct. Cumulative energies integrate instantaneous powers using the trapezoidal rule between consecutive output times.

The results structure duplicates parameters for traceability, stores raw state matrices, exposes convenient aliases for common signals, records conservation metrics, attaches validation output, and includes scalar summaries (final electrical energy, losses, efficiency estimate, drop duration, terminal height).

C Symbolic conventions

- θ, ω — drum angle and angular velocity.
- ω_m — motor angular velocity after all fixed ratios.
- N_{tot} — product of sprocket, gearbox stage, and bevel ratios as implemented.
- P_{elec} — electrical power at the machine terminals into the defined load convention.
- η — per-stage mechanical retention factor applied to shaft power before additional friction subtraction.

Document control

Revision: May 11, 2026. Generated from repository sources. Export to PDF using `pdflatex` as described in `latex/README\_COMPILE.txt`.